TP2 – Rotas na Mina

Algoritmos I
Data de entrega: 03/11

1 Objetivo do trabalho

O objetivo deste trabalho é modelar e resolver o problema computacional descrito a seguir utilizando algoritmos avançados em grafos. Deverão ser desenvolvidas duas soluções corretas para o mesmo problema: uma baseada exclusivamente no paradigma guloso e outra que explore o paradigma de divisão e conquista com o objetivo de reduzir sua complexidade.

Além das implementações, cada aluno deverá comparar as duas abordagens por meio de análise teórica e experimentos. A comparação deverá discutir como o número de linhas, o número de colunas e, principalmente, a quantidade de consultas afetam o desempenho de cada solução.

O código-fonte das duas soluções e uma documentação sucinta, com no máximo seis páginas, deverão ser submetidos via Moodle até a data limite indicada. A documentação deverá apresentar a modelagem do problema, justificar as decisões algorítmicas, analisar as complexidades de tempo e espaço e relatar os resultados experimentais.

2 Definição do problema

Você faz parte da equipe de engenharia responsável pelo sistema de navegação dos robôs de uma mina longa e estreita. Esses robôs transportam equipamentos entre diferentes posições, e a central de operações precisa determinar rotas de custo mínimo para um grande volume de solicitações.

Antes que uma nova versão do sistema seja colocada em operação, a equipe deseja avaliar duas abordagens desenvolvidas a partir de paradigmas distintos. Você deverá construir dois protótipos: um baseado no paradigma **guloso** e outro baseado em **divisão e conquista**. Ambos devem responder corretamente às mesmas solicitações. Além de implementar os protótipos, você deverá comparar seus custos teóricos e seus desempenhos experimentais, identificando como as características da mina influenciam cada abordagem.z

A mina é formada por poucas galerias paralelas, mas se estende por um grande número de setores. Durante uma operação, a central pode receber muitas solicitações de deslocamento entre diferentes posições.

O mapa da mina é representado por uma grade com H linhas e W colunas. Cada célula possui um custo de travessia não negativo. Um robô pode se deslocar entre duas células que compartilham um lado; portanto, movimentos diagonais não são permitidos.

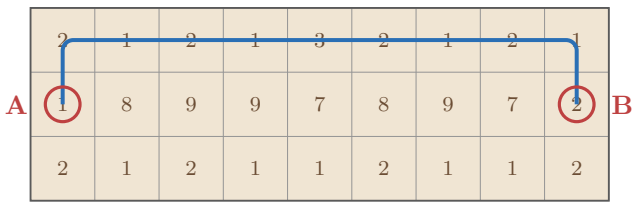


Figura 1: Exemplo de uma mina com poucas linhas e muitas colunas. Os números representam os custos das células. A linha azul ilustra um possível caminho entre as posições de uma consulta; ela não implica que o caminho desenhado seja sempre o de menor custo.

Um caminho é uma sequência de células em que cada par de células consecutivas compartilha um lado. O custo do caminho é a soma dos custos de todas as células visitadas, incluindo as células de origem e de destino. Quando a origem e o destino são a mesma célula, o custo do caminho é o custo dessa célula.

A entrada contém Q consultas. Cada consulta informa duas células da mina, e sua resposta é o menor custo possível entre elas. As consultas não alteram a grade e são conhecidas antes do início do processamento.

A geometria da mina é importante para este trabalho: H **tende a ser pequeno**, enquanto W pode ser grande. Além disso, a central pode receber uma quantidade Q de consultas muito superior a H. Essas características devem ser consideradas no projeto e na avaliação das soluções.

3 Formato da entrada

O programa deverá processar um único caso de teste em cada execução. A primeira linha contém dois inteiros H e W, que representam, respectivamente, o número de linhas e o número de colunas da grade, com

$$2 \leq H \leq 5 \qquad \text{e} \qquad 2 \leq W \leq 10\,000.$$

As H linhas seguintes contêm W inteiros cada. O j-ésimo inteiro da i-ésima linha, denotado por $c_{i,j}$, é o custo de travessia da célula situada na linha i e na coluna j, com

$$0 \leq c_{i,j} \leq 10^6.$$

A linha seguinte contém um inteiro Q, o número de consultas, com

$$1 \leq Q \leq 50\,000.$$

Cada uma das Q linhas seguintes contém quatro inteiros r_1, c_1, r_2 e c_2. Eles indicam que deve ser calculado o custo mínimo de um caminho entre as células (r_1, c_1) e (r_2, c_2). As coordenadas são indexadas a partir de 1 e sempre representam células válidas da grade. Uma mesma consulta pode aparecer mais de uma vez, e suas células de origem e destino podem ser iguais.

Os valores da resposta podem exceder o intervalo de um inteiro de 32 bits. Portanto, as implementações devem utilizar um tipo inteiro de 64 bits para armazenar custos de caminhos.

4 Formato da saída

Para cada consulta, na mesma ordem em que aparece na entrada, imprima uma linha contendo um único inteiro: o menor custo de um caminho entre as duas células informadas.

As duas implementações solicitadas neste trabalho deverão produzir exatamente a mesma saída para uma mesma entrada.

5 Exemplos

Exemplo 1

Entrada

```
3 5
1 1 1 1 1
2 9 9 9 2
1 1 1 1 1
3
1 1 3 5
1 4 3 4
2 2 2 4
```

Saída

```
8
6
21
```

Na primeira consulta, um caminho de menor custo pode descer pela primeira coluna e seguir pela última linha. Na segunda, embora as células estejam na mesma coluna, é mais barato contornar a célula central de custo 9 pela extremidade direita. Na terceira, o melhor caminho deixa temporariamente a segunda linha para evitar parte da região de custo elevado.

Exemplo 2

Entrada

```
2 4
1 100 100 1
1 1 1 1
3
1 1 1 4
1 2 1 3
2 2 2 2
```

Saída

```
6
200
1
```

Na primeira consulta, o caminho de menor custo utiliza a segunda linha para evitar as duas células de custo 100. Na segunda, a ligação direta entre as duas células ainda é a opção mais barata. A terceira consulta possui origem e destino iguais.

6 Soluções que devem ser implementadas

As duas soluções descritas nesta seção são obrigatórias e deverão resolver exatamente o problema definido neste enunciado.

6.1 Solução gulosa

A primeira implementação deverá empregar exclusivamente o paradigma guloso. Ela servirá como solução de referência para a correção dos resultados e como base para a comparação de desempenho.

6.2 Solução com divisão e conquista

A segunda implementação deverá explorar o paradigma de divisão e conquista para processar o conjunto de consultas de maneira mais eficiente. A solução deve dividir o problema em instâncias menores, resolvê-las e combinar adequadamente as informações produzidas.

A implementação e a documentação deverão deixar claros os subproblemas considerados, o caso-base, a etapa de combinação e a razão pela qual todas as consultas são respondidas corretamente.

7 Comparação das soluções

A documentação deverá apresentar uma comparação teórica e experimental entre as duas implementações. A comparação deverá incluir:

1. análise das complexidades de tempo e espaço em função de H, W e Q;
2. descrição dos casos utilizados nos experimentos e do ambiente de execução;
3. medições realizadas com as mesmas entradas para as duas soluções;
4. experimentos em que H permaneça pequeno e W e Q sejam variados;
5. discussão dos resultados, relacionando o comportamento observado às complexidades analisadas.

As medições não deverão ser impressas na saída padrão dos programas. Elas podem ser obtidas por uma ferramenta externa ou enviadas para a saída de erro. Recomenda-se executar cada configuração mais de uma vez e utilizar uma medida representativa, como a mediana dos tempos observados.

A avaliação não dependerá exclusivamente de um limite rígido de tempo. A adequação aos paradigmas solicitados será verificada também pela estrutura do código-fonte, pela justificativa apresentada na documentação e pela comparação experimental. A solução gulosa poderá ser avaliada com instâncias reduzidas, enquanto a solução baseada em divisão e conquista deverá ser capaz de processar as restrições completas.

O aluno é livre para utilizar quaisquer linguagem/ferramenta (ex. Python para gerar gráficos) para realizar a análise experimental.

8 Implementação

Os programas deverão ser implementados em C ou C++ utilizando a biblioteca padrão da linguagem. O código deve compilar e executar nas máquinas do DCC. O arquivo de entrada será fornecido pela entrada padrão e os resultados deverão ser escritos na saída padrão, sem a criação de arquivos adicionais.

Para compilar as soluções utilizando a estrutura disponibilizada, execute `make`. Ao término da compilação deverão ser produzidos dois executáveis:

Executável	Implementação
`tp02-guloso`	solução gulosa;
`tp02-divconq`	solução baseada em divisão e conquista.

Os executáveis poderão ser utilizados, por exemplo, com os comandos:

```
./tp02-guloso < casoTeste01.txt
./tp02-divconq < casoTeste01.txt
```

Ambos deverão respeitar rigorosamente os formatos de entrada e saída definidos neste documento.

9 O que deve ser entregue

Deverá ser submetido um arquivo `.zip` contendo apenas uma pasta chamada `tp2`, com os seguintes itens:

1. **Documentação em PDF:** relatório com no máximo seis páginas contendo identificação (nome e matrícula), introdução, modelagem da grade como grafo, descrição e justificativa das duas soluções, demonstração de corretude, análises de complexidade e comparação experimental;
2. **Implementações:** arquivos-fonte das duas soluções e `Makefile` necessários para produzir os executáveis exigidos;
3. **Instruções dos experimentos:** breve descrição dos comandos, parâmetros ou geradores utilizados para produzir os resultados apresentados no relatório.

A documentação deve explicar as soluções do problema, e não apenas reproduzir ou comentar o código-fonte. Diagramas, pseudocódigo e pequenos exemplos podem ser utilizados quando ajudarem a apresentar a modelagem e os algoritmos adotados.

9.1 Atrasos

Trabalhos poderão ser entregues após o prazo estabelecido, porém sujeitos a uma penalização regida pela seguinte fórmula:

$$\Delta_p = \frac{2^{(d-1)}}{0.32}\% \tag{1}$$

Nesta fórmula, d representa os dias de atraso. Por exemplo, se a nota dada pelo corretor for 70 e o trabalho for entregue com quatro dias corridos de atraso, a penalização será de $\Delta_p = 25\%$ e, portanto, a nota final será $N_f = 70(1 - \Delta_p) = 52.5$. A penalização é exponencial, e seis dias de atraso resultam em uma penalização de 100%.

10 Considerações finais

Assim como em todos os trabalhos desta disciplina, é estritamente proibida a cópia parcial ou integral de códigos, seja da internet, de LLMs ou de colegas. As soluções poderão ser submetidas a ferramentas de detecção de similaridade. Além da correção dos resultados, serão consideradas a adequação aos paradigmas solicitados, a clareza do código, a qualidade da análise e a consistência da comparação experimental.